Academic Cloud Platform Specification

AcademiaQuiz

B.Tech Online Quiz & Real-Time Multi-Device Leaderboard System

A production-grade, responsive academic evaluation and ranking platform engineered for Computer Science & Engineering undergraduates. Features zero mock accounts, instant cross-device cloud synchronization, an 8-semester 40-subject curriculum, and faculty analytics.

Target Curriculum:	B.Tech CSE / IT • BVCEC Odalarevu R23 Autonomous Pattern
Author / Maintainer:	Sriteja Mallula (GitHub: @sritejamallula)
Live Deployment:	https://sritejamallula.github.io/online-quiz/
GitHub Repository:	https://github.com/sritejamallula/online-quiz
System Architecture:	React 18 + TypeScript + Vite 6 + Tailwind 4 + REST Cloud Sync
Release Version:	v2.0 (Real-Time Cloud Sync & Zero Dummy Users)
Document Date:	October 2026

Table of Contents

Section	Title	Focus Area
1.0	Executive Summary & Core Objectives	Mission, Zero-Mock Policy, Scope
2.0	High-Level System Architecture	Client, Service Layer, Cloud REST DB
3.0	Real-Time Cross-Device Cloud Sync Engine	REST Protocol, Polling, Deduplication
4.0	User Experience & System Workflows	Student, Leaderboard, Faculty Dashboards
5.0	Core Data Models & TypeScript Interfaces	Profiles, Attempts, Payloads, Ranks
6.0	8-Semester B.Tech CSE Curriculum Mapping	40 Core Subjects & Syllabus Mapping
7.0	Cloud Persistence API Specification	Endpoints, Methods, Payload JSON
8.0	Local Setup & Production Deployment	Build Pipeline, GitHub Pages Static Hosting

Key Architecture Shift: Traditional static web quizzes store student submissions only in browser local storage (`localStorage`). This completely isolates each user to their individual device, forcing instructors to rely on mock dummy profiles. AcademiaQuiz eliminates all fake accounts, utilizing a live cloud synchronization protocol so that all classmates in a batch interact in a shared, synchronized evaluation ecosystem.

Document Conventions & Terminology

- **Student Profile:** An authenticated undergraduate identity uniquely defined by institutional Roll Number (e.g. `23A91A0501`).
- **Quiz Attempt:** A verified assessment session comprising 10 multiple-choice questions with recorded score, elapsed time, and submission timestamp.
- **Real-Time Sync:** Bi-directional data propagation across disparate client devices via automated 8-second polling, window focus listeners, and cross-tab communication.
- **Zero Dummy Policy:** The operational rule mandating that no synthetic, preloaded, or fallback accounts exist in the application database.

1.0 Executive Summary & Core Objectives

The **AcademiaQuiz** system is an interactive evaluation platform engineered to support modern higher education in engineering. In typical classroom and laboratory environments, instructors and student cohorts face persistent challenges when conducting short-format academic assessments:

- **Device Isolation:** Client-only web applications cannot share state across laptops and smartphones without complex backend infrastructure.
- **Synthetic Leaderboards:** Conventional demonstration systems fill leaderboards with dummy users, obscuring genuine student performance.
- **Curriculum Disconnect:** Generic quiz platforms do not align with university-specific syllabi and unit-level learning objectives.

AcademiaQuiz solves these challenges through four primary pillars:

1.1 Zero Dummy Users Guarantee

No preloaded mock student profiles or artificial scores exist within the database. The initial state is completely clean (`registeredStudents: []`, `quizAttempts: []`). Leaderboards, statistics, and roster views are populated exclusively by actual classmates who open the application on their respective devices and register.

1.2 Real-Time Multi-Device Cloud Synchronization

Through a zero-setup REST persistence engine, every registration and quiz attempt is broadcast to all active student devices and the faculty portal within an 8-second polling cycle. An instant event-driven sync occurs when students submit forms, unlock mobile devices, or refocus browser tabs.

1.3 Academic Rigor & GeeksforGeeks-Style Explanations

Covering 40 core theory subjects across all 8 semesters of the B.Tech CSE curriculum (BVCEC Autonomous R23 pattern), each quiz offers unit-level evaluation with in-depth technical rationale and code snippets upon completion.

1.4 Dynamic Competitive Leaderboard

Ranks students through a hybrid scoring formula accounting for cumulative points and accuracy. Top performers are highlighted on animated 3D metallic Gold, Silver, and Bronze podium blocks, fostering positive peer motivation.

2.0 High-Level System Architecture

AcademiaQuiz utilizes a decoupled client-cloud architecture that combines Single Page Application (SPA) speed with serverless cloud persistence.

Layer	Technologies	Primary Responsibility
Presentation Layer	React 18.3, Tailwind CSS 4, Lucide Icons	Renders light-mode glassmorphic cards, question palette grids, countdown timers, and responsive tables.
State Management	React Context API (`QuizContext.tsx`)	Maintains global state: registered classmates, quiz attempts, current student identity, active semester, and quiz mode.
Synchronization Layer	`cloudSync.ts`, HTML5 `BroadcastChannel`	Manages REST API push/pull operations, background polling timers, deduplication, and cross-tab communication.
Persistence Layer	Cloud REST Backend (`api.restful-api.dev`), `localStorage`	Houses the central JSON payload in the cloud with CORS support, backed by browser offline caching.

Architectural Data Flow

1. **Registration / Attempt Trigger:** A student registers or submits a 10-question quiz on their device.
2. **Local State Mutation:** The React Context state updates immediately, providing zero-latency user feedback.
3. **Cloud Push:** The sync engine packages the updated lists and dispatches an asynchronous HTTPS PUT request to the cloud endpoint.
4. **Cross-Tab Broadcast:** On the local machine, sibling tabs receive instant notification via the HTML5 `BroadcastChannel` API.
5. **Multi-Device Fetch:** Classmates' devices retrieve the latest state during their next 8-second polling interval or immediately upon regaining window focus.

3.0 Real-Time Cross-Device Cloud Sync Engine

The core multi-user synchronization mechanism is implemented in `src/services/cloudSync.ts`. It provides continuous real-time data persistence without requiring a dedicated custom backend server.

3.1 Synchronization Triggers & Event Handlers

Trigger Event	Mechanism	Behavior & Purpose
Background Polling	`setInterval(..., 8000)`	Queries cloud endpoint every 8 seconds to fetch new classmates and attempts.
Window Focus	`window.addEventListener("focus")`	Executes an immediate sync whenever a student switches back to the browser tab or unlocks their phone.
Cross-Tab Sync	`new BroadcastChannel("quiz_sync_channel")`	Shares registration and score changes across multiple tabs on the same computer in < 5 ms.
Mutation Push	Synchronous event call	Pushes data immediately when `loginStudent()` or `submitQuiz()` is invoked.
Manual Refresh	Navbar status pill click	Allows students and instructors to trigger an instant cloud check on demand.

3.2 De-duplication & Conflict-Free Merging

To ensure data integrity when multiple students submit quizzes concurrently:

- **Student Deduplication:** Profiles are keyed by normalized roll number (`rollNumber.trim().toUpperCase()`). If the cloud payload and local cache contain the same roll number, the record with the most recent `registeredAt` timestamp is selected.
- **Attempt Deduplication:** Every quiz submission is assigned a unique UUIDv4 (`attempt.id`). Inbound attempts are merged using a Set-based key lookup, ensuring no attempt is lost or counted twice.

> **Offline Resilience:** If network connectivity drops while a student is completing a quiz, the attempt is safely stored in local browser storage (`btech_quiz_attempts_v2`). As soon as internet connectivity returns, the engine pushes the cached attempt to the cloud.

4.0 User Experience & System Workflows

4.1 Student Registration & Automated Promotion

To begin, students complete a streamlined registration dialog:

- **Full Name & Roll Number:** Used for unique identification and public leaderboard displays.
- **Branch:** Engineering discipline (`CSE`, `AI&DS`, `IT`, `ECE`, `EEE`).
- **Admission Batch Year:** Used by the timeline promotion engine to automatically compute the student's default semester based on elapsed calendar months (e.g. 2024 Batch → Sem 2-1).
- **Active Semester:** Students can manually select their active semester (Sem 1-1 to Sem 4-2) at any time.

4.2 Timed Quiz Interface & Question Palette Grid

- **10-Minute Countdown Timer:** Displays remaining time with visual color shifts (slate to rose pulse under 2 minutes) and automatic submission at 00:00.

  Interactive Question Palette Grid: A 10-button navigation panel displaying question status:

  - **Emerald:** Answered questions.
  - **Amber:** Questions flagged for review.
  - **Slate:** Unanswered / unvisited questions.

- **Submission Modal:** Presents a clear summary of answered, flagged, and unanswered questions prior to final submission.

4.3 Detailed Performance Report & GeeksforGeeks-Style Explanations

Upon quiz completion, students receive immediate pedagogical feedback:

- **Performance Metrics:** Final Score, Accuracy %, Elapsed Time, and Pass/Fail evaluation.
- **Confetti Celebration:** High scores (≥ 80%) trigger dynamic confetti celebration animations.
- **Step-by-Step Technical Rationale:** Each question displays the selected option, the correct option, and a comprehensive explanation explaining the engineering concepts involved.

4.4 Dual-Tab Dynamic Leaderboard

- **Score Leaderboard:** Ranks classmates who have completed quizzes using:
  `Rank Score = Total Points + (Average Accuracy × 0.1)`
  Top 3 scorers are presented on metallic Gold ■, Silver ■, and Bronze ■ 3D podium blocks.
- **All Registered Classmates:** Full live directory of all enrolled classmates across devices with branch, year, roll number, and registration timestamps.

4.5 Faculty Analytics Dashboard

- **KPI Cards:** Enrolled Classmates count, Class Average Accuracy, Pass Rate %, and Active Subjects.
- **Assessment Parameter Controls:** Adjust timer duration, pass mark %, negative marking, and instant feedback.
- **Audit Log:** Chronological log of all submissions across the entire class.

5.0 Core Data Models & TypeScript Specifications

5.1 StudentProfile Interface

```
export interface StudentProfile {
 id: string;               // e.g. "stu_1729431000"
 name: string;             // Full student name
 rollNumber: string;       // Institutional roll number (e.g. "23A91A0501")
 email: string;            // Auto-generated student email
 branch: 'CSE' | 'AI&DS' | 'IT' | 'ECE' | 'EEE';
 admissionYear: number;    // e.g. 2023
 currentSemester: number;  // 1 to 8 (Sem 1-1 to Sem 4-2)
 quizzesAttempted: number; // Count of submitted quizzes
 averageScore: number;     // Percentage average (0 - 100)
 totalPoints: number;      // Cumulative raw points
 registeredAt?: string;    // ISO 8601 creation timestamp
}
```

5.2 QuizAttempt Interface

```
export interface QuizAttempt {
 id: string;               // Unique UUID v4
 studentRoll: string;      // Roll number of the candidate
 studentName: string;      // Candidate full name
 quizId: string;           // Module quiz identifier
 subjectCode: string;      // e.g. "CS2102"
 score: number;            // Points scored (0 - 10)
 totalQuestions: number;   // Standardized to 10 questions
 percentage: number;       // Percentage score (0 - 100)
 passed: boolean;          // Evaluated against pass cutoff
 timeSpentSeconds: number; // Total seconds taken
 completedAt: string;      // ISO 8601 completion timestamp
}
```

5.3 CloudSyncPayload Interface

```
export interface CloudSyncPayload {
 registeredStudents: StudentProfile[];
 quizAttempts: QuizAttempt[];
 lastSyncedAt: string;     // ISO 8601 synchronization timestamp
}
```

5.4 LeaderboardEntry Interface

```
export interface LeaderboardEntry {
 rank: number;
 student: StudentProfile;
 totalScore: number;
 accuracy: number;
 quizzesTaken: number;
 badge?: 'gold' | 'silver' | 'bronze';
}
```

6.0 8-Semester B.Tech CSE Curriculum Mapping

Semester	Code	Theory Subject	Core Curriculum Domains

Sem 1-1	`BS1101`
	`BS1102`
	`CS1101`
	`ES1101`
		Linear Algebra & Calculus
		Applied Chemistry
		C Programming & Problem Solving
		Basic Civil & Mech Engg
			Matrices, Eigenvalues, Multiple Integrals
			Water Tech (EDTA), Nernst, Li-ion Cells
			Pointers, Dynamic Memory, Arrays, Recursion
			Surveying, 4-stroke IC Engines, Boilers

Sem 1-2	`BS1201`
	`BS1202`
	`ES1201`
	`CS1201`
	`HS1201`
		Applied Physics
		Mathematics-II (M-2)
		Basic Electrical & Electronics
		Data Structures
		Communicative English
			Optics, Lasers, Quantum Mechanics
			Differential Equations, Laplace, Vector Calc
			KCL/KVL, 3-Phase, Transformers, Zener, BJT
			Stacks, Queues, Linked Lists, Trees
			Technical Vocabulary, IEEE Report Writing

Sem 2-1	`CS2101`
	`CS2102`
	`CS2103`
	`CS2104`
	`HS2101`
		Discrete Mathematics (DMGT)
		DBMS
		Java Programming
		Advanced Data Structures
		Universal Human Values
			Logic, Eulerian Graphs, Recurrences, Posets
			Relational Algebra, 3NF/BCNF, B+ Trees, 2PL
			OOP, Threads, Exceptions, HashMap internals
			Red-Black Trees, AVL, Bellman-Ford
			Self-Exploration, Natural Acceptance

Sem 2-2	`CS2205`
	`CS2201`
	`CS2202`
	`CS2203`
	`CS2204`
	`HS2201`
		Python Programming
		Theory of Computation (FLAT)
		Computer Organization (COA)
		Operating Systems
		Software Engineering
		Managerial Economics (MEFA)
			Comprehensions, Decorators, Generators, GIL
			DFA/NFA, Pushdown Automata, Turing Machines
			Pipeline Hazards, Cache Mapping, DMA
			CPU Scheduling, Deadlocks, Virtual Memory
			Agile Scrum, Spiral Model, Black Box Testing
			Break-Even Point, Elasticity, Accounting

Sem 3-1	`23AD5T03`
	`23AD6T05`
	`23AD5D03`
	`23CS5S06`
	`23AD5T02`
		Computer Networks
		Machine Learning
		Exploratory Data Analysis
		Full Stack Development
		Artificial Intelligence
			OSI/TCP-IP, Subnetting, 3-Way Handshake, OSPF
			SVM, K-Means, Regularization, Precision/Recall
			Outliers, Pearson Correlation, Chi-Square
			React Virtual DOM, Express, JWT, WebSockets
			A* Search, Minimax Alpha-Beta, CSP

Sem 3-2	`23AD6D06`
	`CS3202`
	`CS3203`
	`CS3204`
	`HS3201`
		Microprocessors & Microcontrollers
		Cryptography & Network Security
		Data Warehousing & Mining
		Cloud Computing
		Ethics & Cyber Laws
			8086 Instruction Set, 8051 Architecture, 8255
			AES, RSA Factorization, Diffie-Hellman, SHA
			OLAP/OLTP, Star Schema, Apriori Algorithm
			IaaS/PaaS/SaaS, Hypervisors, S3, Docker
			IT Act 2000 Section 66, Patents, GDPR

Sem 4-1	`23AD7D09`
	`23CS7T13`
	`23HM7T07`
	`CS4101`
	`CS4102`
		DevOps
		Deep Learning
		HR & Project Management
		Big Data Analytics
		Info Security & Ethical Hacking
			CI/CD, Dockerfiles, Kubernetes Pods, Terraform
			CNNs, Transformer Self-Attention, LSTMs, Adam
			CPM/PERT Critical Path, COCOMO Estimation
			Hadoop HDFS, MapReduce, Spark RDDs, CAP
			SQLi, XSS, Nmap, Burp Suite, CSRF Tokens

Sem 4-2	`CS4201`
	`CS4202`
	`CS4203`
	`CS4204`
	`CS4205`
		Cyber Security & Forensics
		Distributed Systems
		Software Project Management
		Prompt Engg & Generative AI
		Natural Language Processing
			Chain of Custody, Memory Forensics, Volatility
			Raft/Paxos Consensus, Lamport Clocks, RPC
			Risk Exposure, Burn-Down Charts, CMMI Level 5
			Few-Shot, Chain-of-Thought, RAG, LoRA, ReAct
			Stemming, TF-IDF, Word2Vec, BERT Masked LM

7.0 Cloud Persistence API Specification

Parameter	Specification Details
Base Endpoint	`https://api.restful-api.dev/objects/ff808181a09d98f701a15976d9952db0`
Supported HTTP Methods	`GET` (Read live database state), `PUT` (Mutate database state)
Headers Required	`Content-Type: application/json`, `Accept: application/json`
CORS Policy	`Access-Control-Allow-Origin: *` (Direct browser-to-cloud communication enabled)
Serialization Strategy	Database object holds `name` and a nested `data.payload` string containing the stringified `CloudSyncPayload`. This bypasses schema limits on arbitrary nested JSON arrays.

Sample PUT Request Payload

```
{
 "name": "btech_online_quiz_registry_sritejamallula",
 "data": {
   "payload": "{\"registeredStudents\":[{\"id\":\"stu_1729431000\",\"name\":\"Sriteja Mallula\",\"rollNumber\":\"23A91A0501\",\
 }
}
```

8.0 Local Setup & Production Deployment

8.1 Local Development Commands

```
# 1. Clone repository
git clone https://github.com/sritejamallula/online-quiz.git
cd online-quiz

# 2. Install dependencies
npm install

# 3. Start local Vite development server
npm run dev
# Running at http://localhost:5173/

# 4. Compile production build
npm run build
```

8.2 Production GitHub Pages Deployment Pipeline

The application is hosted directly on GitHub Pages from the repository root:

1. Executing `npm run build` generates production assets in `./dist/`.
2. Compiled static assets (`dist/assets/*` and `dist/index.html`) are mirrored to root directory (`assets/` and `index.html`) using relative base paths (`./assets/...`).
3. Commits pushed to `main` branch trigger immediate GitHub Pages deployment at: **https://sritejamallula.github.io/online-quiz/**

> **Document Verification:** This document was generated directly from the live codebase and production deployment of AcademiaQuiz. For latest updates and issues, refer to the official repository at **https://github.com/sritejamallula/online-quiz**.